

OFFSEC

QUEBRA DE SENHA



12

LISTA DE FIGURAS

Figura 1 – Assinatura Digital	7
Figura 2 – Técnica <i>Pass-the-Key</i>	12
Figura 3 – Técnica <i>Pass-the-Ticket</i>	13
Figura 4 – Rainbow Tables	18
Figura 5 – Hydra	20
Figura 6 – John The Ripper	21
Figura 7 – OphCrack	23
Figura 8 – Metasploit Framework	24
Figura 9 – Saída do Hydra no Kali Linux	26

SUMÁRIO

1 QUEBRA DE SENHA	4
1.1 Introdução	4
2 CRIPTOGRAFIA E A FUNÇÃO HASH.....	4
2.1 Criptografia simétrica ou Chave Privada	5
2.2 Criptografia assimétrica ou Chave Pública.....	6
2.3 Assinatura digital	7
2.4 Função hashing	8
3 SEGURANÇA DE SENHAS NO WINDOWS.....	9
3.1 Ataques <i>Pass-the-Hash</i> e <i>Pass-the-Ticket</i>	11
4 SEGURANÇA DE SENHAS NO LINUX	13
4.1 WordLists	15
4.2 Rainbow Tables.....	17
5 SMB O VILÃO DOS USUÁRIOS.....	18
6 FERRAMENTAS DE QUEBRA DE SENHA	19
6.1 Hydra.....	20
6.2 Medusa.....	21
6.3 John the Ripper	21
6.4 OphCrack	23
6.5 Hashcat	23
7 METASPLOIT FRAMEWORK.....	24
8 LAB.....	25
REFERÊNCIAS.....	27

1 QUEBRA DE SENHA

1.1 Introdução

Um dos princípios da Segurança da Informação é a Autenticidade, e é sob a égide desse princípio que usamos comumente logins e senhas nos diversos sistemas que acessamos diariamente, pois se entende que por meio da identificação do usuário mediante login e senha, confirma-se que ele seja quem diz ser.

Filosoficamente, esse conceito é muito interessante, porém, na prática, é infundado, uma vez que a identificação por meio de login e senha não prova que aquela identidade foi realmente utilizada pelo seu detentor. Seria algo muito parecido como em um furto de um carro, no qual o criminoso obtivesse as suas chaves antes de roubá-lo. De posse delas, não precisaria violar os sistemas de segurança do carro, ou seja, ele não é o dono do carro, nem das chaves, mas conseguiu acesso ao veículo de maneira legítima, utilizando a chave correta.

Entender o funcionamento de um ataque de força bruta, no qual se acessa determinado alvo com as credenciais legítimas sem tê-las de antemão valendo-se de diversos métodos, é fundamental para um profissional de Defesa Cibernética. Não só do ponto de vista do ataque, haja vista que esse profissional vai realizar diversos testes de penetração em sistemas e redes e precisará entender bem esses conceitos, mas também do ponto de vista da defesa, uma vez que, por meio dos conhecimentos adquiridos nesta disciplina, será possível implementar medidas de defesa contra ataques de força bruta, sejam elas por meio de tecnologias ou de políticas e conscientização em sua organização.

2 CRIPTOGRAFIA E A FUNÇÃO HASH

A fim de entendermos a concepção de um processo de autenticação que utiliza uma senha para determinado sistema computacional, precisamos, antes, estabelecer os conceitos basilares sobre criptografia e função hash, além de relembrarmos alguns fundamentos da Segurança da Informação. Sob esse ponto de vista, conforme Oliveira (2012), a credibilidade nos sistemas computacionais é definida pela

disponibilidade, integridade, controle de acesso, autenticidade, não repudição e finalmente a privacidade, os quais devem ser de livre compreensão e facilmente perceptíveis ao se efetuar transações computacionais.

- **Disponibilidade** – Garantir que uma informação estará disponível para acesso no momento desejado.
- **Integridade** – Garantir que o conteúdo da mensagem não foi alterado.
- **Controle de acesso** – Garantir que o conteúdo da mensagem somente será acessado por pessoas autorizadas.
- **Autenticidade** – Garantir a identidade de quem está enviando a mensagem.
- **Não repudição** – Prevenir que alguém negue o envio e/ou recebimento de uma mensagem.
- **Privacidade** – Impedir que pessoas não autorizadas tenham acesso ao conteúdo da mensagem, garantindo que apenas a origem e o destino tenham conhecimento.

Assim, é fundamental que técnicas computacionais sejam empregadas para que os requisitos de proteção da informação sejam atendidos. Nesse cenário, apresentam-se os dois tipos básicos de criptografia: a simétrica, ou chave privada, e a assimétrica, ou chave pública.

2.1 Criptografia simétrica ou Chave Privada

Segundo Oliveira (2012), este é o modelo mais antigo de criptografia, em que a chave, isto é, o elemento que dá acesso à mensagem oculta trocada entre duas partes, é igual (simétrica) para ambas as partes e deve permanecer em segredo (privada). Tipicamente, essa chave é representada por uma senha, usada tanto pelo remetente, para codificar a mensagem numa ponta, como pelo destinatário, para decodificá-la na outra.

Essencialmente, quando a origem (ALFA) cifra uma mensagem, ela utiliza um algoritmo de ciframento para transformar o conteúdo em claro da mensagem em texto cifrado novamente. Quando o destino (BRAVO) decifra uma mensagem, ele utiliza o

algoritmo de deciframento correspondente para converter o texto cifrado em uma mensagem clara. Se um intruso (CHARLIE) conhecer o algoritmo de ciframento, ele poderia decifrar uma mensagem cifrada tão facilmente quanto o destino (BRAVO). A solução no uso da criptografia de chave privada propõe que, quando a origem (ALFA) cifra uma mensagem, ela utilize um algoritmo de ciframento e uma chave secreta para transformar uma mensagem clara em um texto cifrado. O destino (BRAVO), por sua vez, ao decifrar a mensagem, utiliza o algoritmo de deciframento correspondente e a mesma chave para transformar o texto cifrado em uma mensagem em claro. O intruso (CHARLIE), por não possuir a chave secreta, mesmo conhecendo o algoritmo, não conseguirá decifrar a mensagem. A segurança do sistema passa a residir não mais no algoritmo e, sim, na chave empregada. É ela (a chave privada) que, agora, no lugar do algoritmo, deverá ser mantida em segredo pela origem (ALFA) e pelo destino (BRAVO).

2.2 Criptografia assimétrica ou Chave Pública

De acordo com Oliveira (2012), este é um modelo de criptografia criado na década de 1970 – pelo matemático Clifford Cocks, que trabalhava no serviço secreto inglês, o GCHQ em que cada parte envolvida na comunicação usa duas chaves diferentes (assimétricas) e complementares, uma privada e outra pública. Nesse caso, as chaves não são apenas senhas, mas arquivos digitais mais complexos (que, eventualmente, até estão associados a uma senha). A chave pública pode ficar disponível para qualquer pessoa que queira se comunicar com outra de modo seguro, mas a chave privada deverá ficar em poder apenas de cada titular. É com a chave privada que o destinatário poderá decodificar uma mensagem que foi criptografada para ele com sua respectiva chave pública.

Para entender o conceito, basta pensar num cadeado comum protegendo um determinado bem. A mensagem é esse bem, e o cadeado, que pode ficar exposto, é a chave pública. Apenas quem tiver uma chave particular (privada) que consiga abrir o cadeado poderá acessar a mensagem. A principal vantagem desse método é a sua segurança, pois não é preciso (nem se deve) compartilhar a chave privada. Por outro lado, o tempo de processamento de mensagens com criptografia assimétrica é muitas

vezes maior do que com criptografia simétrica, o que pode limitar seu uso em determinadas situações.

Essencialmente, o destino (BRAVO) e todos os que desejam comunicar-se de modo seguro geram uma chave de ciframento e sua correspondente chave de deciframento. Ele mantém secreta a chave de deciframento, chamada de “sua chave privada”. Ele torna pública a chave de ciframento, chamada de “sua chave pública”. A chave pública realmente condiz com seu nome. Qualquer pessoa pode obter uma cópia dela. O destino (BRAVO) inclusive encoraja isso, enviando-a para seus amigos ou publicando-a na internet. Assim, o intruso (CHARLIE) não tem nenhuma dificuldade em obtê-la. Quando a origem (ALFA) deseja enviar uma mensagem ao destino (BRAVO), precisa primeiro encontrar a “sua chave pública”. Feito isto, ela cifra sua mensagem utilizando a chave pública do destino (BRAVO), despachando-a em seguida. Quando o destino (BRAVO) recebe a mensagem, ele a decifra facilmente com sua chave privada. O intruso (CHARLIE), que interceptou a mensagem em trânsito, não conhece a chave privada do destino (BRAVO), embora conheça sua chave pública. Mas esse conhecimento não o ajuda a decifrar a mensagem. Mesmo a origem (ALFA), que foi quem cifrou a mensagem com a chave pública do destino (BRAVO), não pode decifrá-la agora.

2.3 Assinatura digital



Figura 1 – Assinatura Digital
Fonte: Google Imagens (2019)

Segundo Oliveira (2012), o sistema de criptografia assimétrica ou de chave pública também é utilizado como um meio de assinatura digital. A pessoa que assina usa sua chave privada para criptografar uma mensagem conhecida, e o texto cifrado pode ser decifrado por qualquer um usando a chave pública dessa pessoa, assim como uma assinatura em papel, que consiste em um bloco de informação adicionado à mensagem, comprovando a identidade do emissor e confirmando quem ele diz ser.

O processo se baseia em uma inversão do sistema, em que o funcionamento da assinatura digital pode ser descrito como: o emissor cifra (ou seja, atesta autenticidade) a mensagem com sua chave privada e a envia, em um processo denominado assinatura digital. Cada um que receber a mensagem deverá verificar a validade da assinatura digital, utilizando para isso a chave pública do emissor, reconhecendo, de fato, que a mensagem não foi adulterada. Como a chave pública do emissor apenas decifra (ou seja, verifica a validade) mensagens cifradas com sua chave privada, obtém-se a garantia de autenticidade, integridade e não-repudição da mensagem, o que é apoiado pela função *hashing*, pois se alguém modificar um bit do conteúdo da mensagem ou se outra pessoa assiná-la em vez do próprio emissor, o sistema de verificação não irá reconhecer a assinatura digital dele como sendo válida.

2.4 Função hashing

Conforme Oliveira (2012), a assinatura digital obtida por meio do uso da criptografia assimétrica ou de chave pública infelizmente não pode ser utilizada, na prática, de forma isolada, é necessário o emprego de um mecanismo fundamental para o adequado uso da assinatura digital. Esse mecanismo é a função *hashing*.

Assim, na prática, é inviável e contraproducente utilizar puramente algoritmos de chave pública para assinaturas digitais, principalmente quando se deseja assinar grandes mensagens, que podem levar preciosos minutos ou mesmo horas para serem integralmente cifradas com a chave privada de alguém, em vez disso é empregada uma função *hashing*, que gera um valor pequeno, de tamanho fixo, derivado da mensagem que se pretende assinar, de qualquer tamanho, para oferecer agilidade nas assinaturas digitais, além de integridade confiável.

Serve, portanto, para garantir a integridade do conteúdo da mensagem que representa, por isso, após o valor *hash* de uma mensagem ter sido calculado por meio do emprego de uma função *hashing*, qualquer modificação em seu conteúdo – mesmo em apenas um bit da mensagem – será detectada, pois um novo cálculo do valor *hash* sobre o conteúdo modificado resultará em um valor *hash* bastante distinto. É justamente por isso que a criptografia adotada para uma senha digitada (que nada mais é que um texto) é a função *hashing*, uma vez que esta garante a integridade da senha utilizada pelo usuário, além de assegurar o controle de acesso, uma vez que o texto em claro da senha não será compartilhado entre outros sistemas.

Seguem algumas características da função hash:

- **Unidirecionalidade:** Conhecida um resumo $h(M)$, deve ser computacionalmente impossível encontrar M a partir do resumo.
- **Compressão:** A partir de uma mensagem de qualquer longitude, o resumo $h(M)$ deve ter uma longitude fixa. O normal é que a longitude de $h(M)$ seja menor do que a da mensagem M .
- **Facilidade de cálculo:** Deve ser fácil calcular $h(M)$ a partir de uma mensagem M .
- **Difusão:** O resumo $h(M)$ deve ser uma função complexa de todos os bits da mensagem “ M ”, se modifica um só bit da mensagem M , o hash $h(M)$ deveria mudar a metade dos seus bits aproximadamente.
- **Colisão:** Será computacionalmente impossível, conhecido M , encontrar outro M' tal que $h(M) = h(M')$. Isso é conhecido como resistência débil às colisões.

Exemplos de algoritmos de cálculo de hash: md4, md5, sha1, sha256, sha512.

3 SEGURANÇA DE SENHAS NO WINDOWS

De acordo com Moraes (2016), as credenciais do Windows podem ser armazenadas de diferentes formas, porém, a mais comum (e padrão dos sistemas Windows desde o Windows NT) é a base de dados *Security Account Manager* (SAM).

Um componente chamado *LSA secrets* armazena cache de credenciais do domínio que são usadas pela *Local Security Authority* (LSA). É uma espécie de armazenamento protegido. A LSA é responsável, dentre outras coisas, pelo logon de usuários no sistema. Credenciais podem ser armazenadas na memória do computador por meio do processo LSASS. As senhas podem ser armazenadas usando hashes LM ou NTLM, cabe ressaltar que o hash NTLM é a mesma chave de entrada para o tipo de criptografia RC4-HMAC, o qual é um hash único e não muda, independentemente da senha escolhida. Apesar de ser um grave erro de segurança, a Microsoft aparentemente implementou dessa forma por questões de compatibilidade com sistemas Windows mais antigos.

LSASS é um acrônimo que vem de *Local Security Authority Subsystem Service* (Serviço de Subsistema de Autoridade de Segurança Local). Nesse caso, pode haver armazenadas senhas com criptografia reversível (*reversibly encrypted passwords*), hashes de senhas (NT ou LM) ou tíquetes do Kerberos (TGT e outros tíquetes de serviço).

Hashes também podem ser encontradas na base de dados dos controladores de domínio do AD, que retém todas as credenciais do domínio por acesso via serviços de diretórios ou no arquivo NTDS.DIT.

Há também a possibilidade de que credenciais de domínio sejam armazenadas localmente, de modo que usuários possam fazer logon de maneira interativa mesmo se o controlador de domínio não estiver disponível. É o caso de laptops e notebooks que são colocados como participantes de um domínio e precisam que seus usuários sejam capazes de fazer logon mesmo que não estejam na estrutura do trabalho. Esse mecanismo é conhecido como *cached credentials*.

Esses são alguns exemplos de como credenciais podem ser furtadas. Esses locais são visados por atacantes que buscam extrair credenciais. Após comprometer um sistema, atacantes geralmente escalam privilégios e tentam obter credenciais desses locais, procurando no disco rígido ou com alguma ferramenta que leia a memória ou faça despejo dela.

3.1 Ataques *Pass-the-Hash* e *Pass-the-Ticket*

Ainda conforme Moraes (2016), *Pass-the-Hash* (PtH) e *Pass-the-Ticket* (PtT) são técnicas que utilizam um *hash* de senha, ou um tíquete do Kerberos, para personificar o usuário que detém a senha para aquele *hash* ou o tíquete. Nesses casos, tanto o *hash* quanto o tíquete funcionam como equivalentes de senha. O protocolo Kerberos especifica operações a serem aplicadas na senha de usuário para transformar uma senha de tamanho variável em uma chave de longo prazo de tamanho fixo, que servirá como entrada para as várias formas de algoritmos de criptografia suportados pela especificação (AES 128, AES 256, DES, 3DES, RC4...).

As chaves geradas podem ser armazenadas no processo LSASS para serem usadas pelo usuário durante a sessão (sessão LSA). Uma vez que essas chaves servem como entrada para os diferentes tipos de algoritmos de criptografia, elas são usadas nesse sentido quando furtadas, como equivalentes de senha. Há várias ferramentas para executar tais ataques. No Kali Linux temos, por exemplo, o Mimikatz, Psh-toolkit, Msvctf, Metasploit PSEXEC module etc.

Moraes (2016) nos traz que um modelo de autenticação padrão, largamente conhecido, é aquele em que o servidor armazena em sua base de dados os *hashes* da senha de usuário. Isso é feito para que, caso a base de dados seja comprometida, o atacante não consiga acesso imediato ao texto claro das senhas de usuários. Em uma arquitetura cliente-servidor, em um processo de autenticação, é comum o cliente passar a senha para o servidor, por meio de um canal seguro preestabelecido, o servidor calcula o *hash* da senha e o compara com o armazenado em sua base de dados, autenticando o usuário caso os *hashes* coincidam. Imagine uma aplicação em que o *hash* da senha de usuário seja calculado no cliente e enviado por meio de um canal inseguro para o servidor, que comparará os *hashes* e o autenticará caso coincidam. Um atacante capturando o tráfego na rede pode obter esse *hash* e repetir a chamada do cliente ao servidor, personificando o usuário e obtendo a autenticação do servidor. Esse exemplo é o tipo de ataque conhecido como *Pass-the-Hash*.

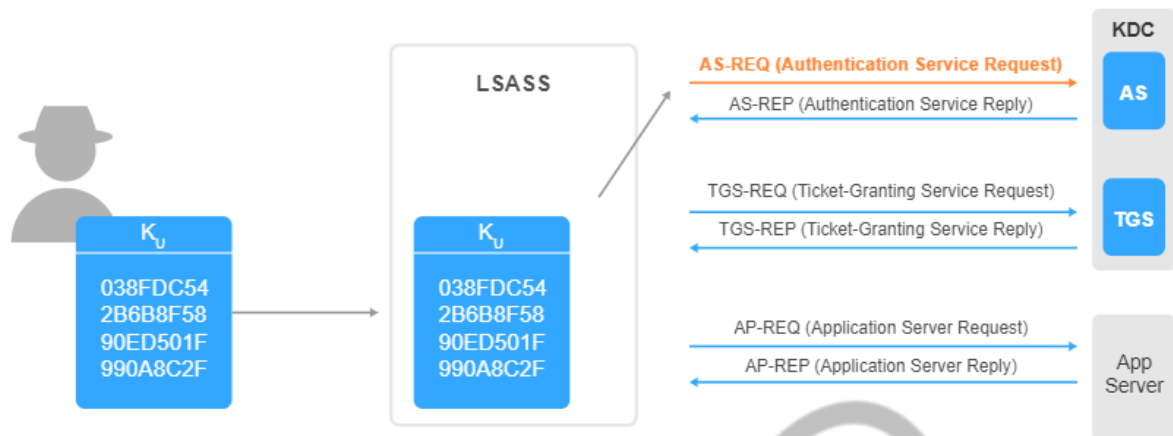


Figura 2 – Técnica *Pass-the-Key*
 Fonte: Google Imagens (2019)

Apresentado na Figura “Técnica *Pass-the-Key*”, como em uma subcategoria ou uma especialização, na técnica *Overpass-the-Hash* ou *Pass-the-Key*, o atacante obtém uma das chaves de longo prazo de um usuário, chaves essas que são derivadas da senha dele, e a injeta em memória, no processo LSASS. Assim, quando a requisição AS-REQ é feita, essa chave será usada para criar um carimbo de tempo cifrado como autenticador.

No ataque *Pass-the-Ticket*, o invasor obtém um tíquete de serviço válido, gerado pelo AS ou TGS, em resposta a uma requisição do usuário. O tíquete obtido pode ser tanto um TGT ou um tíquete de serviço. O tíquete é injetado na LSASS e usado para requisitar outros tíquetes ou acessar o serviço correspondente ao tíquete. Apesar de amplamente conhecido na literatura como PtT, esse tipo de ataque, quando usa o TGT, mereceria um nome especial dentro das subcategorias de ataques PtT. Defende-se isso por ele se tratar de um tíquete especial quando comparado com os tíquetes de serviço e por dar mais poder ao atacante, o poder de requisitar acesso a serviços em nome daquele usuário a quem o tíquete pertence.

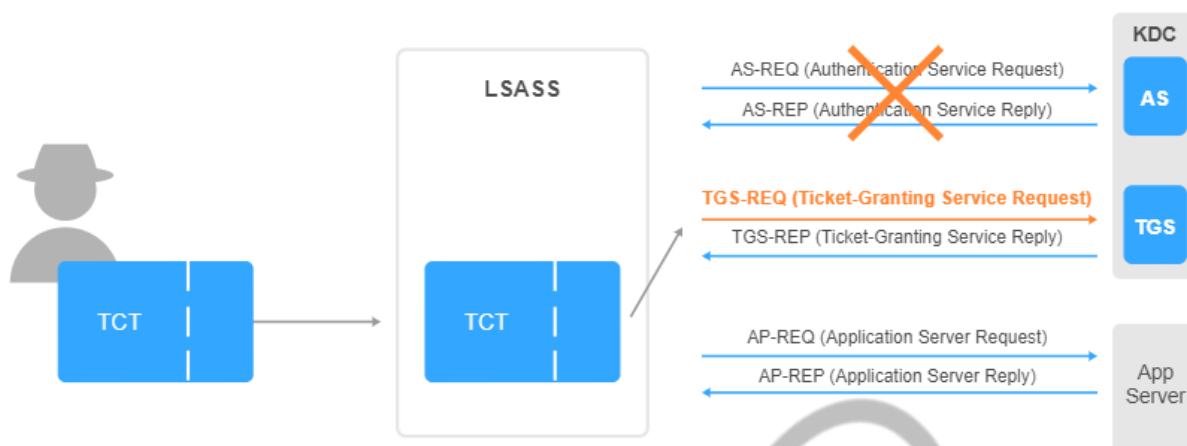


Figura 3 – Técnica Pass-the-Ticket
Fonte: Google Imagens (2019)

4 SEGURANÇA DE SENHAS NO LINUX

Toda conta de usuário em sistema Linux tem uma entrada no arquivo **/etc/passwd**. Esse arquivo contém entradas, uma linha por usuário, que especificam diversos atributos para cada conta. Cada entrada nesse arquivo tem o mesmo formato, com campos separados por "dois pontos":

usuário:senha:UID:GID:NomeReal:/diretório/pessoal:shell

Exemplo de arquivo **/etc/passwd**:

```
root:x:0:0:root:/root:/bin/bash
ftp:x:14:50:FTP                                User:/var/ftp:/sbin/nologin
postgres:x:26:26:PostgreSQL                    Server:/var/lib/pgsql:/bin/bash
fulano:x:500:500:Fulano                        da Silva:/home/fulano:/bin/bash
siclano:x:501:501:Siclano Souza:/home/siclano:/bin/bash
```

OBS.: "x": é a indicação de que essa conta possui hash da senha omitida, no arquivo **/etc/shadow**.

O arquivo **/etc/shadow** armazena as senhas em um formato criptografado, com algumas informações adicionais relacionadas às senhas dos usuários. Contém uma entrada por linha, separada em campos, para cada usuário definido no **/etc/passwd**.

usuário:senha:última alteração:mínimo:máximo:aviso de
expiração:inativo:expiração:

Exemplo de arquivo **/etc/shadow**:

```
root:$6$utLIMf6gY40$Eoe.2M7uqTK1UBtnz3yz4T:14768:0:99999:7:  
daemon*:14715:0:99999:7::  
lp*:14715:0:99999:7::  
shutdown*:14715:0:99999:7::  
ftp*:14715:0:99999:7::  
nobody*:14715:0:99999:7::  
postgres:!:14855::::::  
fulano:$6$nsd3oKxkg0$klEvBac347MZXYfOnrc56ybz:14768:0:99999:7::  
siclano:$6$AaNgoXJSPsv/$cupVfet0y4ciYLpPSWyNQM:14771:0:99999:7::  
beltrano:$6$Qc3AYPlBx0$ikGowHMwzkqRz9QGZzQfwfu:14770:0:99999:7::
```

A string completa com o hash, que representa a senha, é armazenada no arquivo **/etc/shadow** no formato **\$ID\$SALT\$SENHA**. O caractere "\$" (cifrão) delimita cada item da string.

```
nome_do_usuário:$6$eKn9QGMQ$Ofi1OoClVpw/cCbTsD4YUKgfurcQoBxsZ9Tlk5V  
BjTpAjSYFt.M9shPtQVnouNr4/3PRDP/eMqkoWQpuBxsRk1:15362:0:99999:7::
```

Sendo o \$6 o tipo de algoritmo de senha utilizado, conforme segue:

- \$1 = MD5.
- \$2 = Blowfish.
- \$2a = eksblowfish.
- \$5 = SHA-256.
- \$6 = SHA-512.

O \$eKn9QGMQ\$ é o *salt* (traduzido do inglês: sal) implementado no hash de senhas dos sistemas Linux de maneira randômica para aumentar o nível de complexidade e segurança do *hash*. O restante (em roxo no exemplo acima) seria a senha após ser calculada com o *hash* escolhido (SHA-512 neste caso).

Apesar de ser uma implementação mais segura do que nos ambientes Windows, existe uma forma de burlar essa implementação e chegar à senha em claro do usuário.

A aplicação **unshadow**, que utiliza ataques do tipo dicionário, separa os campos da entrada do arquivo **/etc/shadow** comparado ao **/etc/passwd** e tenta comparar o hash (por meio do *password guessing* – técnica de comparação de hash) com as entradas do dicionário utilizado, chegando, assim, à senha em claro.

4.1 WordLists

Os dicionários de senha, ou *wordlists*, (também chamados simplesmente de “dicionários”) são arquivos em texto contendo possíveis senhas, uma em cada linha, para ataques do tipo força bruta (*Brute Force*), nos quais seriam usadas técnicas de *password guessing* e *password cracking*.

A diferença básica entre essas duas técnicas de *brute force* são bastante claras. No *guessing*, o atacante que não possui o hash da senha legítima do usuário realiza o ataque por meio de tentativas de login, ocorrendo o sucesso quando o hash da senha usada pelo atacante combina com o hash da senha legítima do usuário. Já no *cracking*, o atacante explora vulnerabilidades da implementação do sistema de autenticação tentando logar-se com o hash da senha legítima do usuário, a qual foi capturada por outros métodos, em vez de utilizar uma senha propriamente dita.

Um exemplo claro de utilização da técnica de *password cracking* seria no ataque *Pass-the-Hash* em sistemas Windows, em que, de posse do hash da senha do usuário, o atacante conseguirá realizar o login.

No caso de ataques força bruta utilizando a técnica de *password guessing*, e sabendo que esse tipo de ataque consiste em tentativas, o sucesso do ataque está diretamente ligado ao dicionário utilizado. Quanto mais eu souber sobre o meu alvo, mais “palavras” eu poderei inserir em meu dicionário a fim de realizar um ataque bem-sucedido. Logo, para um ataque com maior probabilidade de sucesso, é necessário um trabalho bem-feito de Engenharia Social e Inteligência sobre meu alvo.

Existem algumas ferramentas no Kali que podem nos auxiliar na criação do nosso dicionário, tais como:

a) CUPP.

Uma ferramenta interativa, escrita em Python, que é amplamente usada para criação de *wordlists* por meio de análises de Engenharia Social. Um fator interessante sobre a CUPP é que ela possibilita a inserção de caracteres especiais de maneira automatizada nas senhas geradas.

Sua forma de uso é simples:

```
cupp -i
```

b) CeWL.

O CeWL é uma ferramenta interessante para geração de *wordlists* por meio de análise de inteligência em sites. Basta passar à ferramenta um endereço que ela cria a *wordlist* com base nas palavras do site informado, desta forma:

```
cewl <opções> URL  
cewl -m5 -d2 -w pqdt.txt www.sejalivre.org
```

Em que:

- -m informa o tamanho mínimo da senha;
- -d informa a quantidade de links que o CeWL vai acessar do site informado para capturar palavras.

c) CRUNCH.

O Crunch é uma das ferramentas mais famosas para a criação de *wordlists*. Com ela, é possível criar senhas randômicas, como senhas numéricas, por exemplo, senhas baseadas em anagramas, senhas que possuam somente determinados caracteres etc.

Seu uso é basicamente este:

```
crunch <min> <max> <opções> <caracteres> > <arquivo>  
  
crunch 5 5 admin > dicionario.txt  
  
crunch 1 1 -p abcde > dicionario.txt  
  
crunch 1 1 -p ab cd ef > dicionario.txt
```



```
crunch 9 9 -f /usr/share/crunch/charset.lst lalpha -t @@admin@@ >
dicionario.txt
```

O Kali possui alguns dicionários já instalados, os quais foram confeccionados com base na língua inglesa ou em outros fatores, como, por exemplo, a lista de senhas mais usadas no último ano. Esses dicionários podem ser encontrados em:

```
/usr/share/john/password.lst
/usr/share/metasploit-framework/data/wordlists/password.lst
/usr/share/wordlists/rockyou.txt
```

Se você é jogador de CTF – (*Capture The Flag*), ou se interessa pelo assunto, saiba que a rockyou é a lista mais usada nesses challenges e, com certeza, sua flag poderá ser encontrada nela.

4.2 Rainbow Tables

As Rainbow Tables são tabelas pré-computadas utilizadas para quebrar hashes de senhas. As tabelas têm o intuito de economizar tempo/processamento da máquina em troca de espaço em disco.

Suponha que temos uma função de hash de senha H e um conjunto finito de senhas P . O objetivo é pré-calcular uma estrutura de dados que, dada qualquer saída h da função hash, possibilita localizar um elemento p em P tal que $H(p) = h$, ou determinar que não exista tal p em P . A maneira mais simples de fazer isso é calcular $H(p)$ para todo p em P , mas armazenar a tabela requer $\Theta(|P|n)$ bits de espaço, em que n é o tamanho de uma saída de H , que é proibitivo para grandes $|P|$.

As Rainbow Tables são uma técnica para diminuir esse requisito de espaço. A ideia é definir uma função de redução R que mapeie os valores de hash de volta para os valores em P . Observe, no entanto, que a função de redução não é realmente um inverso da função hash, mas uma função diferente com um domínio e um codomínio trocados. Ou seja, com base em tabelas predefinidas de cálculos de hash para senhas já conhecidas, é possível chegar à senha legítima do usuário apenas juntando essas partes conhecidas dos hashes.

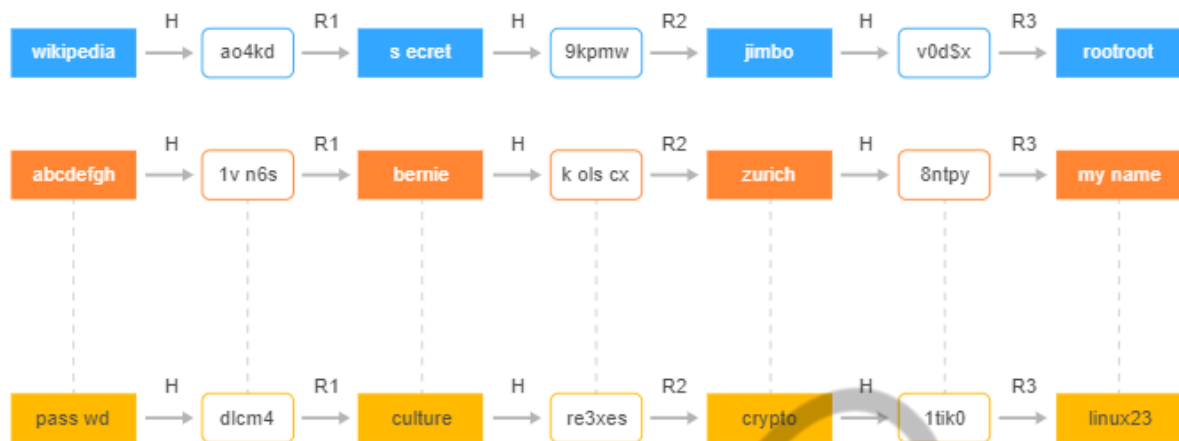


Figura 4 – Rainbow Tables
Fonte: Google Imagens (2019)

Essa é uma técnica amplamente usada para ataques em sistemas Windows e *Rainbow Tables* para versões específicas do Windows e podem ser baixadas livremente na internet.

5 SMB O VILÃO DOS USUÁRIOS

Para ser bem-sucedido em ataques do tipo *password cracking* é necessário que se capture o hash da senha legítima do usuário e, apesar de poder ser realizado de diversas formas, a mais comum delas é pela exploração do protocolo SMB (*Server Message Block* – protocolo de rede da camada de aplicação que é usado para fornecer acesso compartilhado a arquivos, diretórios, impressoras e portas seriais).

O serviço de compartilhamento de arquivo, por padrão, é executado na porta 445. Se mal configurado, permitindo uma NULL SESSION, por exemplo, pode ser útil para enumerar usuários e compartilhamentos do sistema-alvo.

No Kali, existem algumas ferramentas para essa tarefa, tais como:

a) ENUM4LINUX.

Ferramenta específica para enumeração por meio do protocolo SMB e pode ser usada assim:

```
enum4linux [options] ip
```

```
enum4linux -U 192.168.0.33 → Enumera usuários
```

```
enum4linux -S 192.168.0.33 → Enumera compartilhamentos
```

b) RPCCLIENT.

Ferramenta para enumeração por meio do protocolo RPC (*Remote Procedure Call* – protocolo de comunicação entre processos que permite o acesso a processos em máquinas remotas). Assim como o SMB, o RPC possui vulnerabilidades que permitem a enumeração de usuários do sistema-alvo.

O rpcclient pode ser usado assim:

```
rpcclient -U <qualquer usuário> <ip>
```

```
rpcclient -U Teste 192.168.0.33
```

```
enumdousers → Usuários
```

c) NMAP.

Famosa ferramenta de enumeração e *information gathering*, por meio dos seus scripts, o NMAP consegue enumerar usuários e compartilhamentos SMB dessa forma:

```
nmap --script smb-enum-users.nse -p445 <ip> Usuários
```

```
nmap --script smb-enum-shares.nse -p445 <ip> Compartilhamentos
```

6 FERRAMENTAS DE QUEBRA DE SENHA

Nesta seção, iremos apresentar as principais ferramentas de quebra de senha presentes no Kali Linux.

6.1 Hydra



Figura 5 – Hydra
Fonte: Google (2019)

O Hydra é uma ferramenta famosa que realiza ataques de dicionário a serviços. Seu uso é basicamente assim:

```
hydra -l <login> -p <senha> <IP> <serviço> -t <tasks>
```

A seguir, um exemplo de uma tentativa única (-t 1) com o usuário root (-l root) e a senha 12345 (-p 12345) no endereço 10.0.2.2 no serviço ssh:

```
hydra -l root -p 12345 10.0.2.2 ssh -t 1
```

Podemos passar um dicionário de logins e senhas com o parâmetro -L e -P:

```
hydra -L <arquivo_login> -P <arquivo_senha> <IP> <serviço>
```

```
hydra -L login.txt -P senha.txt 10.0.2.2 ssh -t 4
```

O Hydra também possui uma interface gráfica que pode ser chamada com o comando:

```
xhydra
```

6.2 Medusa

Assim como o Hydra, o Medusa realiza ataques de dicionário a serviços. Seu uso é basicamente:

Para listar os módulos:

```
medusa -q | more
```

Comando:

```
medusa -h <ip> -u <usuario> -p <senha> -M <modulo>
```

```
medusa -h 10.10.100.25 -u admin -p 1234 -M ftp
```

Lista de IP, usuários e senhas

```
medusa -H <ips> -U <usuarios> -P <senhas> -M <modulo>
```

```
medusa -H hosts.txt -U users.txt -P pass.txt -M smbnt
```

6.3 John the Ripper

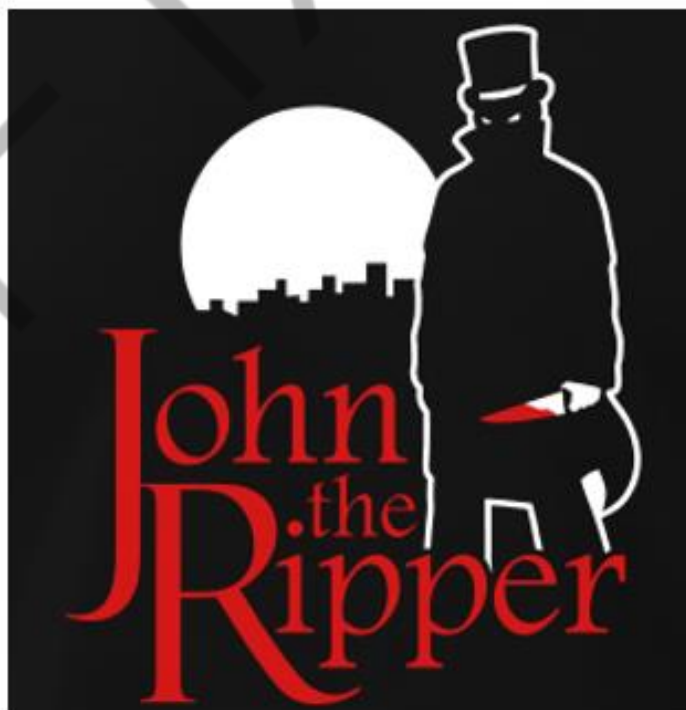


Figura 6 – John The Ripper
Fonte: Google imagens (2019)

O John the Ripper (o nome da ferramenta faz alusão a um famoso psicopata do século 19, Jack o Estripador) é uma das ferramentas mais usadas em pentest, sendo um utilitário que faz quebra de senhas por três modos:

- **WordList** → Ele tenta por uma Wordlist, testando as combinações de senha/usuário.
- **Single Crack** → Ele tenta quebrar a senha usando as informações de login.
- **Incremental** → Sendo o modo mais robusto no John the Ripper, ele tentará cada caractere possível até encontrar a senha correta. Por esse motivo, é indicado o uso de parâmetros com o intuito de reduzir o tempo de quebra.

No caso do modo incremental, é necessário ter o arquivo contendo o hash da senha do usuário (SAM no Windows ou /etc/shadow no Linux).

Seu uso é basicamente:

```
john <arquivo> --wordlist=dicionario.txt  
john <arquivo> --single  
john <arquivo> --incremental  
john <arquivo> --wordlist=dicionario.txt --format=NT
```

As senhas quebradas são armazenadas no arquivo:

```
<home_do_usuario>/.john/john.pot
```

Assim como o Hydra, o JtR também tem uma Interface Gráfica, que pode ser chamada pelo comando:

```
johnny
```

6.4 OphCrack

O OphCrack é uma ferramenta nativa do Kali Linux e específica para uso com Rainbow Tables. Com uma interface gráfica bem intuitiva, o OphCrack é ideal para realizar ataques de *brute force* em sistemas Windows.

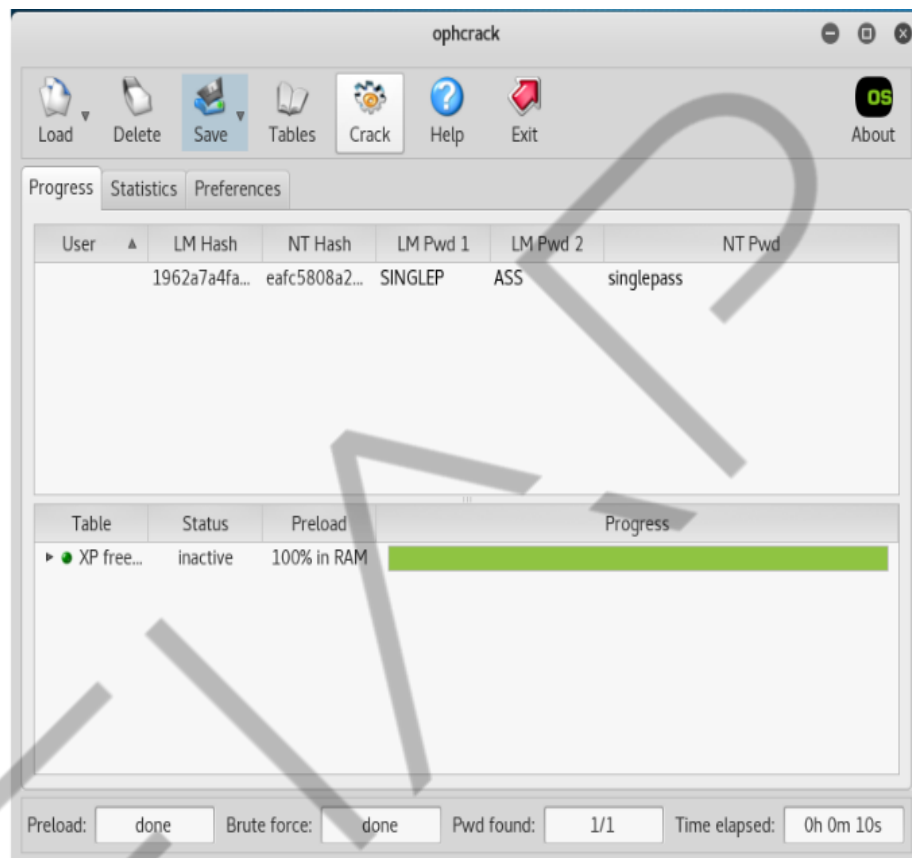


Figura 7 – OphCrack
Fonte: Google Imagens (2019)

6.5 Hashcat

Também nativo do Kali, o Hashcat é uma ferramenta específica para realizar ataques do tipo *Password Cracking*. Uma de suas *features* mais interessantes é a possibilidade de se utilizar o processador da placa de vídeo para dar mais performance ao ataque.

Seu uso é basicamente:

```
hashcat -m <tipo do hash> -a <tipo do ataque> -o <arquivo de saída> <arquivo de hash> <dicionário>
```



```
service postgresql start  
  
msfconsole  
  
use <auxiliar>  
  
auxiliary/scanner/ssh/ssh_login  
  
auxiliary/scanner/ftp/ftp_login  
  
auxiliary/scanner/smb/smb_login  
  
auxiliary/scanner/telnet/telnet_login  
  
show options  
  
set <parametros>  
  
run
```

8 LAB

Você, Analista de Segurança da VortSec Solutions, empresa especializada na realização de testes de segurança em aplicações web, foi selecionado para participar do pentest do sistema de um dos clientes. Como escopo do pentest, as informações a seguir foram passadas pelo gerente do projeto:

- URL do alvo: `attackdirect.samsclass.info`;
- Dicionário: `rockyou.txt`;
- Usuário da aplicação: `root`.

Sua tarefa será realizar um *brute force* em um formulário de login no site do cliente, utilizando o Hydra. Para tal, utilize sua máquina virtual do Kali Linux.

Após o carregamento do sistema, abra um terminal e siga os seguintes passos.

a) Primeiro, iremos descompactar a *wordlist* `rockyou` em seu Kali:

```
cd /usr/share/wordlists  
  
gzip -d rockyou.txt.gz
```

b) Feito isso, podemos iniciar o ataque. Execute o comando a seguir para testar a aplicação utilizando o Hydra:

```
hydra -l root -P rockyou.txt attackdirect.samsclass.info -t 1 http-get
```

Qual foi o resultado? Conseguiu quebrar a senha?

Esta é uma aplicação vulnerável que aceita vários parâmetros como senha válida. Caso a saída do seu Hydra tenha sido parecida com a Figura “Saída do Hydra no Kali Linux”, parabéns, você conseguiu quebrar a senha do usuário root.

```
root@kali:~/brute# hydra -l root -p password attack.samsclass.info http-get /brute0/
Hydra v7.4.2 (c)2012 by van Hauser/THC & David Maciejak - for legal purposes only

Hydra (http://www.thc.org/thc-hydra) starting at 2013-09-06 14:14:38
[DATA] 1 task, 1 server, 1 login try (l:1/p:1), ~1 try per task
[DATA] attacking service http-get on port 80
[80][www] host: 199.188.72.153 login: root password: password
1 of 1 target successfully completed, 1 valid password found
Hydra (http://www.thc.org/thc-hydra) finished at 2013-09-06 14:14:42
root@kali:~/brute#
```

Figura 9 – Saída do Hydra no Kali Linux
Fonte: Elaborado pelo autor (2019)

REFERÊNCIAS

MORAES, F. L. R. **Honeyhashes e Honeytickets**: Detecção de Ataques Pass-the-hash e Pass-the-ticket em Redes de Domínio Active Directory do Windows. 2016. Disponível em: <[\(PDF\) HONEYHASHES E HONEYTICKETS \(researchgate.net\)](#)>. Acesso em: 10 fev. 2021.

OLIVEIRA, R. R. **Criptografia Simétrica e Assimétrica**: os principais algoritmos de cifragem. 2012. Disponível em: <<http://www.ronielton.eti.br/publicacoes/artigorevistasegurancadigital2012.pdf>>. Acesso em: 10 fev. 2021.